

BUILD-API.md

19. Versionnement

Version : V1

Statut : Référentiel officiel du versionnement des API de Vitala

19.1 Objectif

Cette section définit les règles de versionnement applicables à toutes les API de Vitala.

Le versionnement permet :

- d'assurer la stabilité des intégrations ;
- de préserver la compatibilité avec les applications existantes ;
- de faciliter l'évolution des fonctionnalités ;
- de limiter les ruptures de service lors des mises à jour.

Toutes les API doivent suivre la politique décrite dans ce document.

19.2 Principes

Le versionnement repose sur les principes suivants :

- stabilité des contrats ;
 - compatibilité ascendante lorsque cela est possible ;
 - évolutions progressives ;
 - documentation systématique ;
 - coexistence temporaire des versions lors des migrations.
-

19.3 Format des versions

Les API sont versionnées par numéro majeur.

Le numéro de version apparaît dans l'URI.

Exemples :

```
/api/v1/users  
/api/v1/flashses  
/api/v1/missions
```

Le format officiel est :

```
/api/v{major}/{resource}
```

19.4 Compatibilité ascendante

Une évolution est considérée comme compatible lorsqu'elle :

- ajoute un nouveau champ optionnel ;
- ajoute un nouvel endpoint ;
- améliore les performances sans modifier le contrat ;
- corrige un bug sans changer le comportement attendu.

Ces évolutions ne nécessitent pas une nouvelle version majeure.

19.5 Changements incompatibles

Une nouvelle version majeure est obligatoire lorsqu'une évolution :

- supprime un endpoint ;
- modifie le format d'une réponse ;
- change la signification d'un champ ;
- supprime un champ utilisé par les clients ;
- modifie une règle métier ayant un impact sur le contrat API.

Ces changements doivent être annoncés et documentés avant leur déploiement.

19.6 Dépréciation

Avant la suppression d'une version :

- la version est marquée comme dépréciée ;
- les développeurs sont informés ;
- une période de transition est définie ;
- une documentation de migration est publiée.

Aucune version ne doit être supprimée sans préavis.

19.7 Coexistence des versions

Pendant une période de migration, plusieurs versions peuvent coexister.

Exemple :

```
/api/v1/flashses  
/api/v2/flashses
```

Chaque version doit rester indépendante et correctement documentée.

19.8 Documentation

Chaque version doit disposer de sa propre documentation comprenant :

- les endpoints disponibles ;
 - les différences avec les versions précédentes ;
 - les fonctionnalités ajoutées ;
 - les fonctionnalités dépréciées ;
 - les guides de migration.
-

19.9 Journal des évolutions

Chaque version doit être accompagnée d'un historique des modifications (Changelog) indiquant :

- les nouvelles fonctionnalités ;
- les corrections ;
- les améliorations ;
- les changements incompatibles.

Le journal doit être conservé pendant toute la durée de vie de la version.

19.10 Tests

Toute nouvelle version doit être validée par :

- les tests unitaires ;
- les tests d'intégration ;
- les tests de régression ;
- les tests de compatibilité.

Les anciennes versions encore supportées doivent continuer à être testées.

19.11 Références croisées

Cette section est liée à :

- BUILD-API.md (Principes communs)
- BUILD-API.md (Conventions REST)
- API-ERROR-CATALOG.md
- Domain Model
- API Contracts

Les règles de versionnement doivent rester cohérentes avec l'ensemble de l'architecture documentaire.

19.12 Gouvernance

Toute création d'une nouvelle version majeure doit être :

- approuvée par l'architecte logiciel ;
- documentée ;
- planifiée ;
- communiquée aux équipes concernées.

Les décisions de versionnement doivent être justifiées afin de limiter la fragmentation des API.

19.13 Conclusion

Le versionnement garantit la pérennité des API de Vitala.

En définissant des règles claires de compatibilité, de dépréciation et de migration, la plateforme peut évoluer de manière maîtrisée tout en assurant la continuité des services pour les applications clientes, les partenaires et les futures intégrations.